PhotonicsAI — AI-Driven Science and Autonomous Laboratories

Homework 4 (with 5): From Prototype to Product — the PhotonicsAI Lab Assistant

Name: Erik Mirzoyan

Code: `https://github.com/10mirzoyan71/photonicsai-lab`

What it is. Homeworks 1–3 built three agent systems: physics-informed modelling of the Silverbox oscillator (HW1), human-in-the-loop focusing of light through a scattering medium on a simulated bench (HW2), and an agent that drives a real lab camera through MCP (HW3). This project puts them under one roof as a **laboratory with a director and teams**, and makes it a web application: a person asks the lab a question in plain words, approves steps on the real camera with a button, does what needs hands (blocking the light), and watches the camera, the team reports and the results while the agents work. The new parts are:

- **two more levels of hierarchy**: a director above the three homework systems, which now work as teams;
- **questions passed up the hierarchy**: a team's question to the person reaches the person through the director, also when the process is restarted between the question and the answer;
- **a backend/frontend application**: the lab engine, the camera's MCP server and the model backend behind a web frontend (Streamlit) with approval and action cards, a camera panel, verified results, a token meter, a replay mode, and runs that can be continued or retried;
- **more accurate results**: only VERIFIED numbers reach the report, focusing is reported against the far field as well as against the background ring (the lesson of Homework 2), and the modeling team reproduces the Homework 1 test RMSE exactly;
- **four new instrument tools** (black level, ADC bit depth, temperature, TIFF export; Homework 3, section (g)), which the instrument team uses on its own;
- **a token budget**: every model call is counted, and the design keeps a run to about 1 % of the weekly usage limit of a Claude subscription.

1. The hierarchy

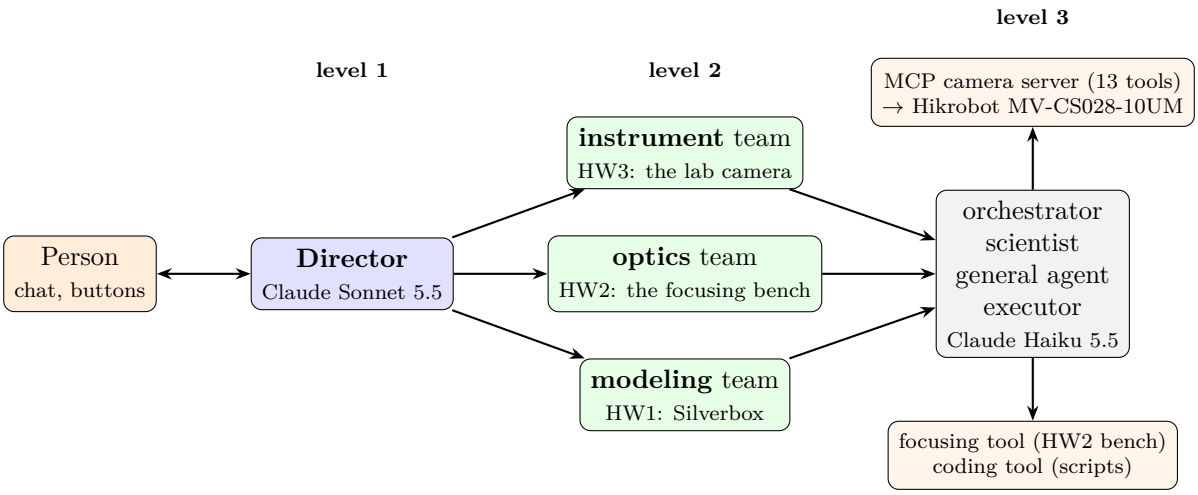


Figure 1: The three levels. The person talks only to the director; each team is the agent graph of Homeworks 1–3 with its own tools; the specialists inside a team call the tools.

level	role	budget
1 director	splits the request into team tasks, passes VERIFIED numbers from one team to the next, stops for the person before a task changes the real camera, writes the final report from VERIFIED records only	6 decisions
2 teams	**instrument**: the lab camera behind the HW3 MCP server (exposure, gain, ROI, black level, ADC bit depth, temperature, captures, time series, frame analysis, TIFF) and a coding tool; **optics**: the simulated HW2 bench with the focusing tool, whose camera model can take measured numbers; **modeling**: the HW1 Silverbox record, the identified Duffing equation and the kept network, with a coding tool	8 / 6 / 6 decisions
3 specialists	orchestrator (decides), scientist (theory and plan), general agent (critique), executor (the only one with tools); inside the instrument team a hardware turn: decide → command → measure → decide	12 steps per hardware turn

2. How the levels work together

The director is a LangGraph `StateGraph` with the nodes `director`, `human` and one node per team, on a thread checkpointed in SQLite. Each decision is a structured output (`DirectorDecision`: the next team or *human* or *finish*, a task, a question, why it stops, the pending step, a reason), so it can only name teams that exist. A task must be self-contained: the team does not see the director's dialogue, so the director copies into it every number it needs from earlier VERIFIED records (e.g. the measured read noise for the optics team). Code, not the model, enforces the rules: *finish* is refused while no team has returned a VERIFIED record; the person is asked for approval only before an instrument-team task and at most once per run (a second request runs the task directly); there is no approval stop before the final report; and after the budget the run stops with what it has.

A team is the Homework 3 agent graph built with the team's tools and prompts (`agent_v3.build_graph`). It runs on its own checkpointed thread (`<run>-<k>-<team>`) and in a separate Python thread, so it is an independent graph and not a subgraph of the director's node. Teams without hardware get a decision schema without the hardware role, so their orchestrator cannot choose it. A team returns its last executor record that is VERIFIED (the numbers a tool printed), and only these records go back to the director.

A question from a team (in a hardware turn: "please block the light") pauses the team's graph with an interrupt. The director's team node then pauses the director with the same question, so the person answers everything in one place; the answer resumes the team with `Command(resume=answer)`. After every resume LangGraph runs the node again from the start and `interrupt()` returns the answers already given in order; the node therefore first replays one interrupt per answer the team has already received and only then passes on the next question. Because both levels are checkpointed, a paused run can be continued from a new process (a test restarts between the question and the answer), and *Retry* runs on from the last checkpoint after an error.

The model backend is the Claude Code CLI in print mode with its own tools switched off (`claude -p --output-format json --tools "" --json-schema ... --effort low`), logged in with a Claude subscription, so no API key is needed; the executor's tool calls come back as JSON and are run by the project's own tools. Any LangChain chat model can be used instead (`make_backend`).

3. The application: backend and frontend

Backend (no user interface, usable from any frontend or a script): `lab/lab_assistant.py` (the director, the teams and `LabRun`: start, resume, retry, pending question, usage, save), `lab/app_core.py` (one worker thread per run with a job queue, the list of saved runs, frames and figures), the camera's MCP server (`hw3/mcp_server/camera_server.py`, a separate process over stdio) and the model backend (`claude_cli.py`). Every run is a folder (`lab/runs/<thread>/`: `run.json`, `transcript.json`,

checkpoints, the teams' files), so a run survives a restart and can be replayed. **Frontends**: the web app `lab/app.py` and the terminal `lab/run_lab.py`, both on the same `LabRun` API. `python -m streamlit run lab/app.py` opens the web app; it reads the backend's state every second while the agents work.

- **Chat**: the request, the director's decisions, each team's report with its VERIFIED record, and the team's internal work (orchestrator, scientist, executor, every camera command and its result) in a box per task.
- **Approval card** (Approve / Refuse with a reason) and **action card** (Done, or an answer in words) in place of typed replies, so a typing slip cannot be read as a refusal (it happened in the first terminal run).
- **Camera panel**: the newest frame the instrument team captured, with its exposure, gain and levels, and the last MCP call. **Verified results**: the teams' records and the figures they wrote. **Tokens**: model calls, output tokens and the CLI's cost estimate of this run, by model.
- **Side bar**: simulated camera (room light or laser speckle) or the real Hikrobot camera; whether the director may ask; the models; the saved runs. **Replay** steps through a finished run with the camera frames in sync (the fallback for a demonstration without the camera or without model calls); a paused run, also one started in the terminal, can be continued in the app.

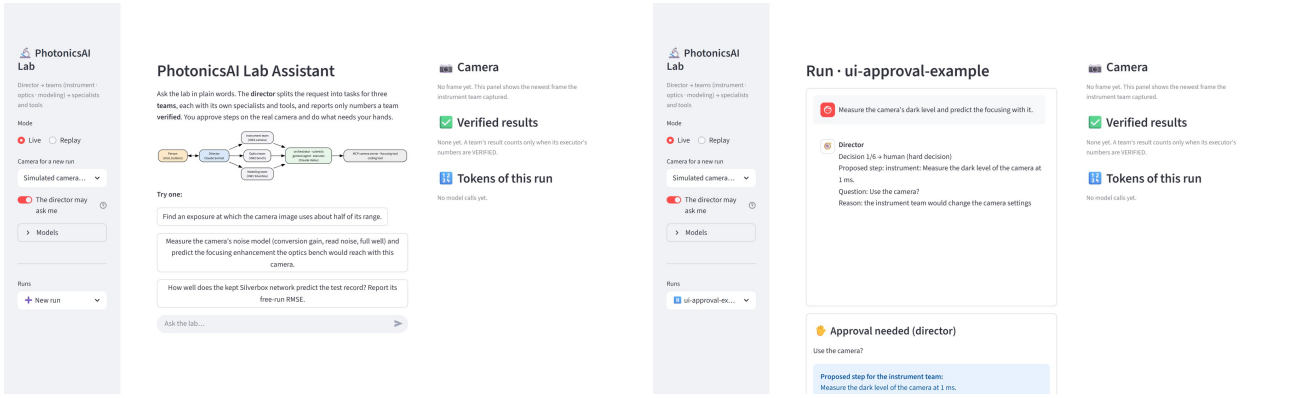


Figure 3: Left: the start page with the hierarchy and three example requests. Right: an approval card (here from an offline test run with scripted models): the proposed step for the instrument team, Approve and Refuse.

4. Runs

The two camera runs used the simulated camera behind the same MCP server as the real one; the director ran on Claude Sonnet 5.5 and the teams on Claude Haiku 5.5, both with low effort. The runs are in `lab/runs/` (transcript, events, token usage, the teams' folders with `calls.jsonl`, frames, scripts and figures).

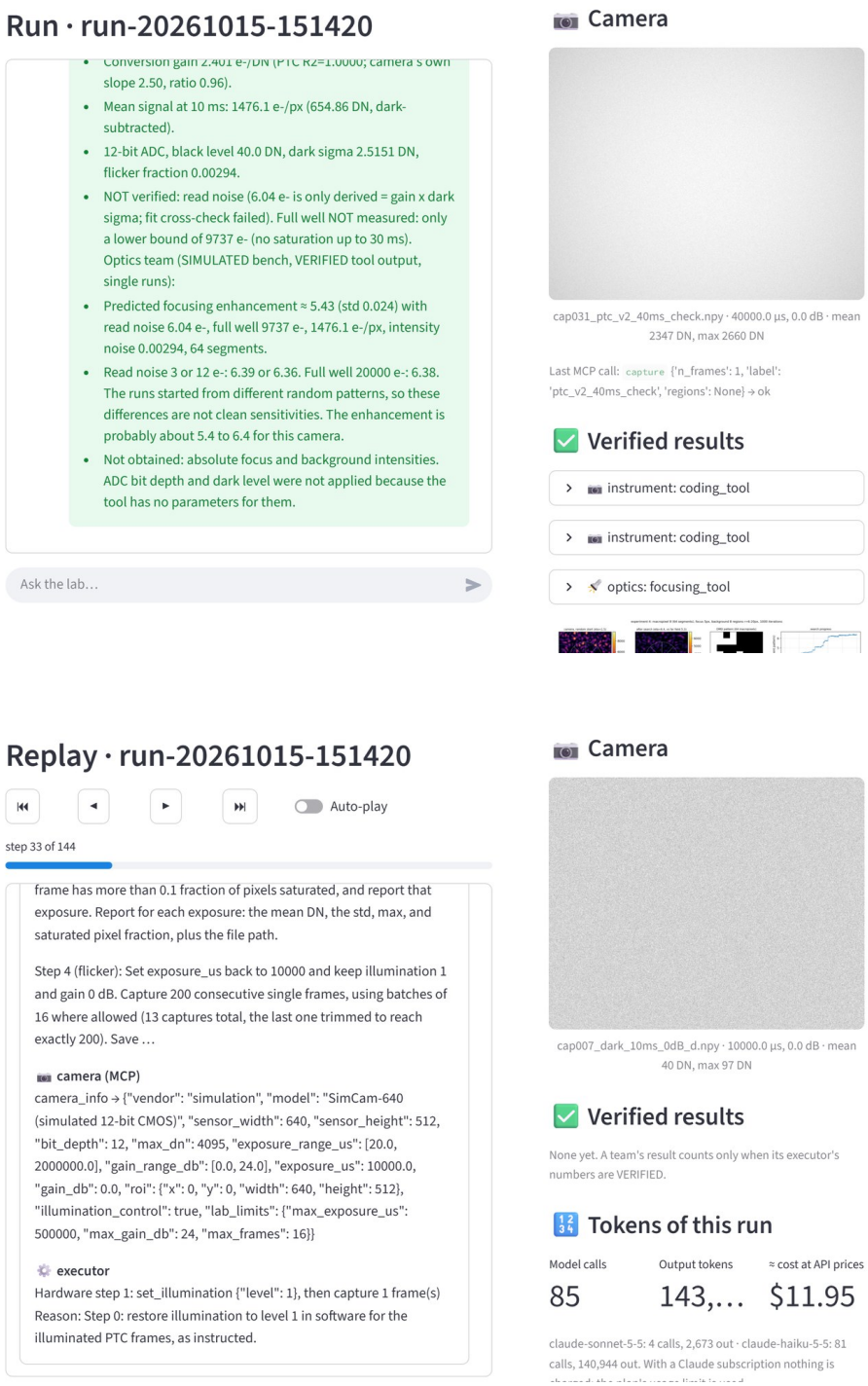


Figure 2: The app (side bar cut off). Top: the run of section 4 after it finished: the director's final report, the last camera frame with its settings and the last MCP call, the teams' VERIFIED records. Bottom: the replay of the same run at step 33, inside the instrument team's work (the executor's hardware steps and the camera's answers through MCP), with the dark frame captured at that step.

run	request and result	model calls	output tokens
`first`	*Find an exposure at which the camera image uses about half of its range.* Instrument team: 34 200 µs at 0 dB, mean 2012 DN = 49.1 % of 4095, no saturated pixel, checked by an analysis script on 5 frames; the exposure set back to its first value afterwards.	24 (5 + 19)	32 971
`run-20261015-151420`	*Measure the camera's noise model and predict the focusing enhancement the optics bench would reach with this camera.* Instrument team (two tasks): conversion gain 2.401 e^-/DN (true 2.40), dark noise 2.52 DN, full well ≥ 9737 e^- (true 10 000), read noise 6.04 e^- (true 6.0) marked *not verified* by the team; optics team with this camera model: enhancement 5.3–5.7 against the far field (5.4–6.4 against the background ring) at 64 segments. 14.5 minutes.	85 (4 + 81)	143 617
`modeling-check`	*How well does the kept Silverbox network predict the test record? Report its free-run RMSE.* (director without stops) Modeling team: free-run RMSE 0.422 mV on the 1024 test samples, 0.70 % of the test output's standard deviation, equal to the value the metric itself returns (Homework 1: 0.422 mV); nothing retrained.	18 (2 + 16)	28 508

In the second run the instrument team set the ADC to 12 bits and the black level itself before measuring, switched the light off with `set_illumination` for 50 dark frames, and ran a photon-transfer series analysed by its own scripts. Its first request to the person ("cover the lens") was answered in words through the action card ("this is the simulated camera: switch its light off with set_illumination 0"); the director's prompt now says this itself. The final report keeps measured and derived numbers apart, names the team for each and states that everything was simulated. The CLI estimated $11.95 at API prices; with the subscription nothing is charged, and the plan's usage meter rose by about 1 % of the weekly limit.

The modeling run is the second try of that request. In the first, the team searched for the data file, re-implemented the metric and finally reported no RMSE, partly because the model sometimes tried to call the coding tool directly instead of listing it in its structured output. The modeling executor now gets the five lines that evaluate a saved model, and the CLI backend answers such a slip with a reminder of the protocol and lets the model try again; the second try returned the verified 0.422 mV in 18 calls.

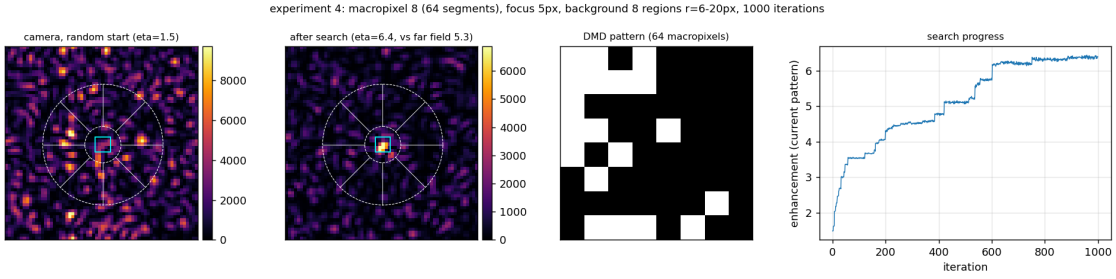


Figure 4: The optics team's last experiment in that run (simulated HW2 bench with the measured camera model): speckle at the random start, the focus after 1000 iterations (enhancement 6.4, far field 5.3), the DMD pattern of 64 segments, and the search progress.

5. Hardware and the live demonstration

The instrument team is the Homework 3 system: the camera's MCP server with 13 tools, the Hikrobot connector through the MVS SDK, the hardware turn, and the lab limits (500 ms exposure, 24 dB gain,

16 frames per capture). On the real camera the director asks the person once before the instrument team changes its settings, and the team asks the person whenever a measurement needs hands (blocking and unblocking the light). The live demonstration on 18 October runs the app with the lab camera (*REAL camera* in the side bar): a short request, the approval card, the camera commands through MCP and the frames in the camera panel; the longer run of section 4 is shown in the replay mode.

Lab check on 18 October (real camera). `python hw3.py check-camera` passed on the MV-CS028-10UM: the new black-level, ADC and ROI tools work on the device, and with the 12-bit ADC a frame has 213 distinct levels against 56 with 8 bits; this model does not expose `DeviceTemperature` (MVS error 0x80000106), so `read_temperature` returns an error on it. Then the app ran on the real camera (run `run-20261018-103704`): request "Find an exposure at which the camera image uses about half of its range", one approval, captures at 4000, 14 000 and 16 400 μs; the final frame has a mean of 2124.4 DN = 0.519 of full scale and no saturated pixels (VERIFIED from the saved frame), with 12 model calls (2 Sonnet, 10 Haiku) and 7520 output tokens.

6. Tests

Without model calls or a camera (scripted models, the real MCP server on the simulated camera):

- `lab/tests/test_lab_offline.py`: the director's approval; an instrument hardware turn whose question ("block the light") is passed up and answered at the director; a restart of the process between two answers; the optics team with the measured camera model; the final report with both VERIFIED records.
- `lab/tests/test_app_offline.py`: the app's worker thread through the same flow (started while the run is still loading, as the page does); the page itself with Streamlit's `AppTest`: the start page, a finished run, the replay controls, and a paused run continued in the page with its approval card.
- `cd hw3 && python hw3.py test`: the simulated camera, the MCP server with the new tools, the agent graph.

7. The token budget

The Claude subscription has a 5-hour and a weekly usage limit, shared with every other use, so the design keeps a run small: Haiku for the teams (81 of the 85 calls of the long run), low thinking effort (36 % fewer output tokens per call in a test), short prompts and reports, a decision budget per level, and at most one approval stop. The meter in `claude_cli.py` adds up every call of the process; each run keeps its own share, also across processes, in `transcript.json`, and the app shows it.

8. Relation to Lecture 5 (multi-hardware integration)

principle of the lecture	in the lab assistant
multi-setup orchestration: a coordinator plans, each setup keeps control of its own devices	the director gives tasks; each team owns its tools (only the instrument team reaches the camera)
agent-to-agent messages: task IDs, typed inputs, status, structured results	structured decisions (Pydantic), one thread ID per team task, VERIFIED records as the results
a remote model plans; deterministic limits stay outside the model	the MCP server refuses what is out of the camera's range or the lab limits (500 ms, 24 dB, 16 frames); the director's guards are code
triggering and records	software trigger (each frame exposed after the settings are applied); every call is logged with its time and duration in `calls.jsonl`
error handling and safe recovery; never hide partial completion	checkpoints at every step, Retry from the last one, restored camera settings after a server restart; the report marks what is not verified
reserve shared resources before dispatch	one run at a time holds the camera; the app closes idle runs before a new one starts

9. Limits and next steps

The lab has no laser for this project, so the optics team's focusing runs on the simulated bench with a camera model measured on the real camera; the next step is the lab's DMD in front of the camera, as a fourth team or as a tool of the optics team, so that the HW2 search runs on hardware. No small language model was fine-tuned; the backend already takes any LangChain chat model, so a local model could take over a team. The director's tasks and the teams' analysis scripts come from language models and can be wrong; the system therefore reports only numbers that tools printed, keeps the derived ones apart, and logs every camera command.

Run it (Python 3.11–3.13): `python -m venv .venv`, `.venv\Scripts\python.exe -m pip install -r requirements.txt`, the Claude Code CLI logged in, and for the real camera Hikrobot MVS with the camera on USB 3 (MVS client closed). Then `python -m streamlit run lab/app.py`, or in the terminal `python lab/run_lab.py --thread t1 --camera hikrobot --request "..."`.